AL SALAM HOSPITALS

Digital Patient Concierge Avatar

Concept, Technical Approach & Build Options — Final Report



Concept direction: a branded, bilingual digital human at the lobby/reception desk (Emirati-style persona, hospital-branded dress).

Companion to: Digital Patient Concierge MVP Development Pack, v1.0

A.1 Purpose and How to Read This Report

This report consolidates the team's brainstorming and technical evaluation of the Digital Patient Concierge Avatar into one document, restated against the structure, scope, and guardrails already agreed in the MVP Development Pack (v1.0). It is organized in five parts:

Part	Content	Read this if…
A — Concept & Approach	Concept direction, confirmed scope, RAG, data control, guardrails, and the orchestration decision this report exists to resolve	You need the full picture and the approved boundaries
B — Option 1	Detailed build plan: Alexa + ElevenLabs + HeyGen, with architecture, integration steps, demo scope, and cost	You want the fastest path to a working pilot
C — Option 3	A self-hosted, open-source alternative memory layer (Graphiti temporal knowledge graph) with worked examples	You're evaluating the RAG/data-control angle
D — Option 4	A broader set of avatar-rendering vendors beyond HeyGen, compared on latency, cost, and persona fidelity	You're choosing the rendered "face" of the assistant
E — Master Action Plan	Every open decision and next step from Parts A–D, deduplicated into one owner/timeline list	You're running Sprint 0 and need the checklist

("Option 2," a LangGraph-based multi-agent orchestration framework, is evaluated alongside Option 1 in Section A.10 rather than as its own part — it is still under investigation and has not yet reached the same build-plan detail as Options 1, 3, and 4.)

A.2 Concept Direction

The working concept is a branded, bilingual digital human stationed at the lobby/reception desk, presented on a kiosk display and mirrored on the website. Early visual concepts show a culturally appropriate Emirati-male persona in hospital-branded dress, positioned at the Al Salam Hospitals reception entrance, with an on-screen bilingual (Arabic/English) welcome panel offering "Schedule a Visit" and "Patient Services" as first actions.

This is consistent with the MVP Pack's Experience Layer (Section 3.1) and the "Welcome and language" capability (Section 2.1): language selection up front, no personal data collected at that stage, and a clear path into the core service journeys.

A.3 Reference Products Reviewed

The team reviewed existing digital-human products to benchmark look, feel, and interaction quality. The product judged closest to the target experience is Humanizing (humanizing.com) — used as a visual and interaction quality bar for this build, not as a vendor recommendation.

Recommendation: treat Humanizing as a reference benchmark inside the Vendor/API Evaluation Matrix (Section 14 of the MVP Pack) rather than a default choice — scored against the same weighted criteria (Arabic/English quality, security, integration, avatar experience, reliability, cost, governance) as any other avatar/speech vendor.

A.4 Confirmed MVP Service Scope

The brainstorming scope matches the MVP Pack's in-scope capabilities (Section 2.1) exactly. Restating it here with the intended delivery mechanism for each:

Service	Delivery mechanism	MVP Pack reference
Doctor search, hospital directions, approved service/price inquiry	Internal REST APIs (Section 10.2)	Section 2.1, Section 10
Appointment request or booking	MCP (Model Context Protocol) tool calling into the scheduling workflow	Section 2.1, US-005

Service	Delivery mechanism	MVP Pack reference
WhatsApp and live-agent handoff	APIs (Sections 10.2: /handoffs, /whatsapp/handoff)	Section 2.1, Flow D

Using MCP for the appointment workflow specifically is the one addition versus the original pack, and is why a multi-agent/orchestration layer is part of the discussion in Sections A.9–A.10.

A.5 Retrieval-Augmented Generation (RAG)

Confirmed: the solution must include a RAG layer over approved hospital content (doctor directory, department/location data, services and prices, FAQs, policies), exactly as specified in Section 9 of the MVP Pack. RAG answers non-transactional questions; live data (availability, bookings, patient-specific results) must continue to come from the deterministic workflow APIs, never from RAG.

A.6 Data and Integration Control

Reconfirmed as a hard requirement: all patient data and system integrations remain under Al Salam Hospitals' control, routed back through hospital-owned tools and APIs — consistent with the private/hybrid architecture in Section 3.1 and the deployment split in Section 3.3. Licensed components (avatar rendering, speech, and any orchestration/foundation-model service) sit behind governed interfaces and receive only the minimum data needed for a given turn.

A.7 Phased Rollout

Rollout sequencing matches Section 2/3 of the MVP Pack: pilot in one selected lobby or outpatient area first, then scale to additional channels and departments, and only later extend into AR wayfinding, VR patient education, and voice-agent services.

A.8 New Capability Flagged: Wayfinding

Wayfinding (turn-by-turn or map-based guidance) was flagged as new relative to the original pack's “directions” capability, which returns static location/route information (Section 2.1, /departments/{id}/directions). True wayfinding is the natural bridge toward the AR indoor-navigation phase (pack appendix, Section 8) and should be scoped as a Should-later item (Epic E8), unless Operations wants a lightweight, non-AR version (step-by-step text/QR directions) added to the MVP backlog.

A.9 Agent Orchestration Approach

To coordinate multiple specialized tasks (doctor search, wayfinding, pricing, appointments, handoff) behind one conversational front end, the team proposes an agent-orchestration pattern — a coordinating layer that routes a patient's request to the right specialized tool or sub-agent, rather than one monolithic prompt handling everything. This maps directly onto the MVP Pack's existing “AI orchestration” layer (Section 3.1) — a refinement of how that layer is built, not a new layer.

A.10 Technical Implementation Paths Under Evaluation

Four candidate ways to build the orchestration/voice/avatar/memory stack have now been identified. None is finalized; all should be validated against the same journeys (doctor search → appointment request → handoff) before a build decision is locked in at Milestone M1 (Section 5.1 of the MVP Pack).

Option	One-line summary	Detail
1 — Alexa + ElevenLabs + HeyGen	Composed stack of licensed, mature components — fastest path to a demo	Part B
2 — LangGraph	Open-source multi-agent orchestration framework, still under investigation	Table below
3 — Graphiti (open-source graph memory)	Self-hosted temporal knowledge graph as an alternative/complement to vector RAG	Part C
4 — Avatar rendering alternatives	Broader vendor set for the rendered "face" component alone	Part D

Option 1 vs. Option 2 — the core orchestration decision

Option 1 (Alexa + ElevenLabs + HeyGen) is the fastest path to a working demo since each component is a mature, licensable service, but does not natively provide multi-agent coordination — routing a single patient request across doctor search, wayfinding, pricing, and appointment sub-tasks with shared state. Option 2 (LangGraph) targets that coordination gap directly, structuring the logic as explicit, auditable agent/tool nodes, but leaves avatar/speech integration, licensing, and hosting undefined.

Dimension	Option 1: Alexa + ElevenLabs + HeyGen	Option 2: LangGraph
Multi-agent orchestration	Gap — needs a coordination layer built on top	Native strength — purpose-built for this
Speed to first demo	Faster — licensed, mature components	Slower — more custom build work
Avatar / speech readiness	Ready out of the box (HeyGen / ElevenLabs)	Not yet defined — to be paired with a rendering/speech vendor
Fit with guardrail model (Sec. 8)	Depends on the added coordination layer	Strong natural fit — explicit, auditable node logic
Status	Feasible now, gap identified	Under investigation

A pragmatic middle path is worth testing: use Option 1's licensed avatar/speech components for the experience layer while adopting LangGraph (or an equivalent framework) purely for the AI-orchestration layer behind it — keeping the architecture split the MVP Pack already calls for (Section 3.1: licensed avatar/speech, hospital-built orchestration). Options 3 and 4 are independent of this decision: Option 3 upgrades the memory/RAG layer and Option 4 broadens the avatar-rendering choice, and either can pair with whichever orchestration path wins here.

A.11 Analytics and Monitoring

Consistent with Section 3.2 of the MVP Pack, all conversation transactions should flow into a single Audit Log / Analytics / Monitoring layer — giving Operations and IT one dashboard view of volume, escalations, unanswered questions, and system health, rather than separate views per component.

A.12 Team Model Alignment

The brainstorming confirms the Section 4 team model from the MVP Pack as the right structure to execute this — having Backend/Integration, Frontend/Kiosk, AI/RAG, DevOps, Security, and Arabic content roles in place from day one is what makes any of the technical paths above executable within the 90-day window.

A.13 Non-Negotiable Guardrails

Reconfirmed without change from Sections 2.2 and 8.1 of the MVP Pack:

Never in this MVP: Diagnosis, medication advice, autonomous clinical triage, full EMR access, AR/VR production rollout, facial recognition.

A consolidated list of every open decision from every part of this report — owners and target dates — is in Part E.

PART B

Option 1 — Detailed Implementation Plan

Alexa + ElevenLabs + HeyGen

B.1 Objective of This Plan

Section A.10 named Option 1 as the fastest path to a working demo, with one known gap: no native multi-agent coordination. This plan makes that option concrete enough to build: which component does what, how they connect, what a small proof-of-concept would cover, and what it would cost and feel like (latency) in the lobby. It intentionally does not try to solve the multi-agent gap — the point of the small demo is to confirm the gap is real and see exactly where it bites, before investing in Option 2 or the hybrid path.

B.2 What Each Component Owns

Component	Role in the stack	Ownership
Alexa	Conversation/voice orchestration: channel handling (kiosk, web, WhatsApp), flow logic, intent routing, calling out to hospital APIs and to the speech/avatar services	Licensed platform, hospital-configured flows
ElevenLabs	Speech layer: Arabic/English speech-to-text and text-to-speech	Licensed API
HeyGen LiveAvatar	Avatar rendering: lip-synced, gesturing video stream over WebRTC	Licensed API
Hospital backend	RAG/knowledge base, Doctor/Directions/Price APIs, Appointment MCP tool, WhatsApp/handoff APIs, audit log	Hospital-owned (Sections 3.1, 10 of MVP Pack)

B.3 Architecture Diagram

Alexa sits as the single orchestration hub in the middle: it is the only component that talks to every other piece. ElevenLabs and HeyGen sit either side as licensed, swappable services; the hospital backend stays entirely on the hospital's side of the boundary (Section 3.1). All five components write events into the shared Audit Log / Analytics / Monitoring layer (Section 3.2).

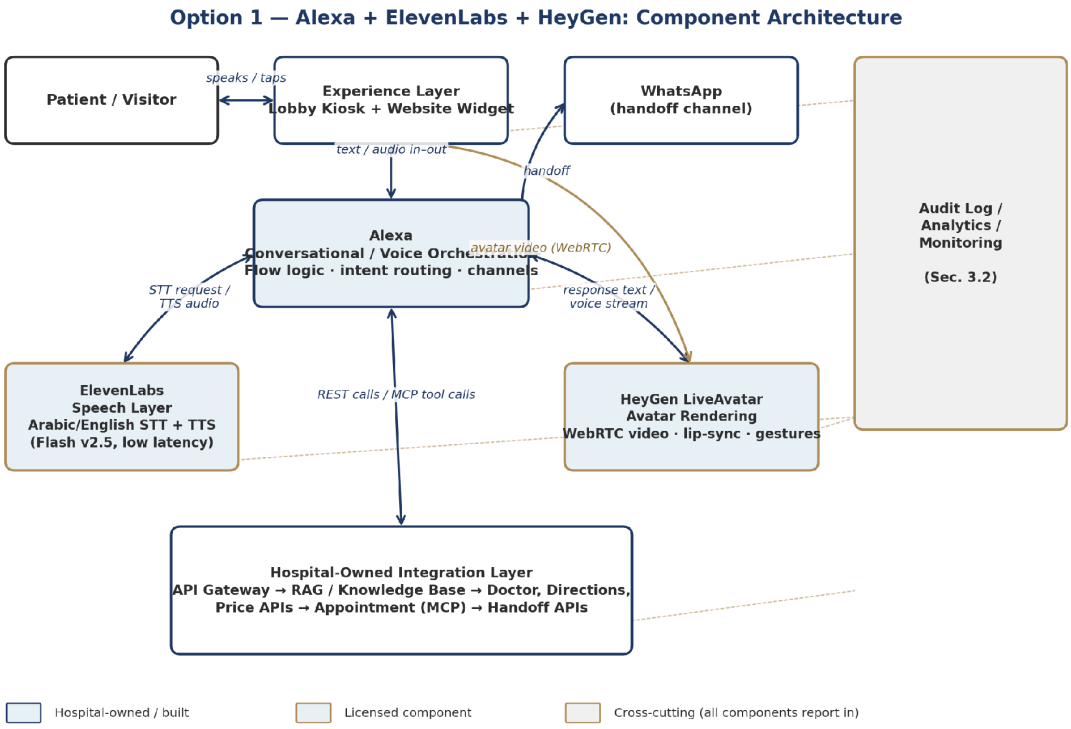


Figure B.1 — Option 1 component architecture: hospital-owned pieces (navy) vs. licensed components (gold).

B.4 What Happens on One Conversation Turn

A single patient turn end-to-end — asking for clinic directions — traced across every component. This is also the walkthrough the demo (Section B.6) needs to reproduce live.

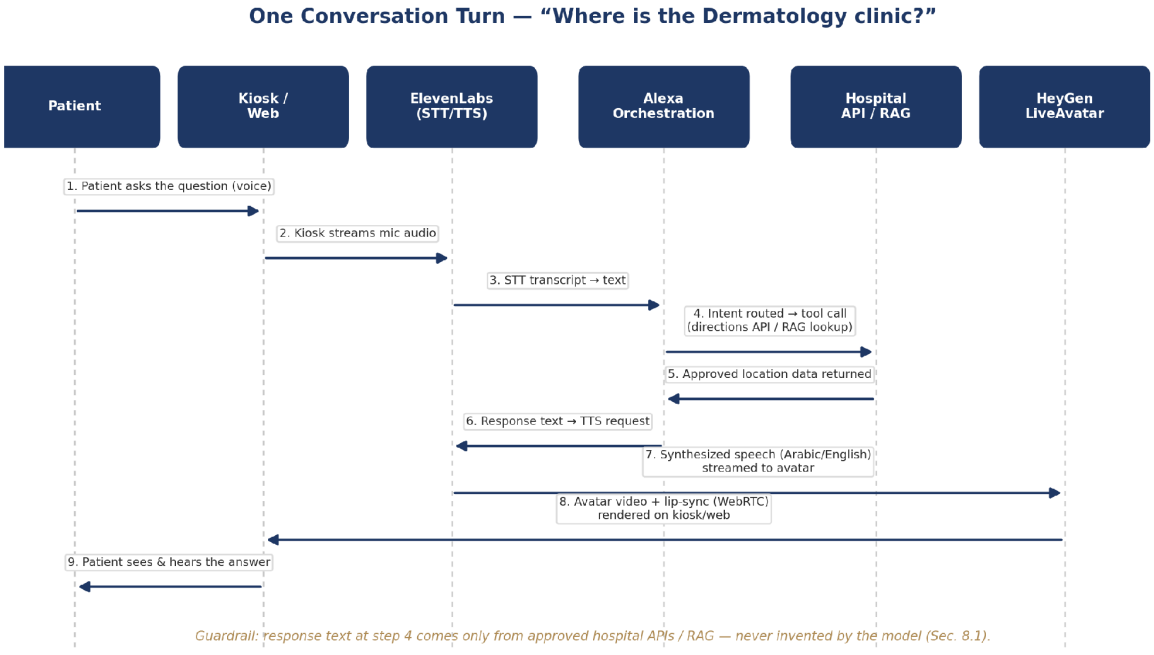


Figure B.2 — Turn-by-turn flow for a single patient question, Kiosk → answer.

Note where the guardrail sits: the orchestration layer (step 4) only ever passes back content retrieved from the hospital's RAG/API layer. Alexa, ElevenLabs, and HeyGen never originate facts — they route, voice, and render them.

B.5 Integration Steps

B.5.1 Alexa Setup

- Stand up an Alexa workspace; configure the kiosk/web channel and (later) WhatsApp as a second channel, so the same flow logic serves both.
- Build the core flow visually (no-code flow builder): welcome → language selection → intent menu → sub-flows for doctor search, directions, price inquiry, appointment request, handoff.
- Use Alexa's external-request / webhook nodes to call the hospital's mock APIs first (Section 10.2 — /doctors, /departments/{id}/directions, /services/{id}), matching the pack's "mock APIs from Day 1" principle.
- Use a JavaScript node to format API responses into clean, patient-facing sentences before they are sent to ElevenLabs for voicing.
- Log every turn (channel, language, intent, API called, outcome) out to the hospital's audit/analytics endpoint.

B.5.2 ElevenLabs Integration

- Use the Flash v2.5 model for both directions (STT and TTS) — ElevenLabs' lowest-latency model (sub-500ms end-to-end), which matters for a lobby kiosk where silence feels broken quickly.
- Configure Arabic (Saudi Arabia / UAE) and English voices; keep the same two voice IDs consistently mapped to the avatar persona.
- Call the API over streaming/WebSocket rather than one-shot requests, so speech starts playing before the whole sentence is generated.
- Store the API key only on the hospital-side backend or Alexa's server-side node — never in kiosk-side/browser code (Section 11.1's secrets-vault requirement).

B.5.3 HeyGen LiveAvatar Integration

- Create/upload the branded Emirati-male persona as a HeyGen avatar; select LiveAvatar's real-time streaming mode (WebRTC, via LiveKit) rather than pre-rendered video.
- LiveAvatar can accept ElevenLabs-generated audio directly (native support for eleven_flash_v2_5 and eleven_multilingual_v2 voices) — simplifying the pipeline by letting HeyGen call ElevenLabs internally for the response leg.
- Pass approved hospital content in as LiveAvatar's "knowledge base" parameter only for tone/persona grounding — never as a substitute for the hospital's own RAG layer (Section 9).
- Budget for LiveAvatar's credit-based pricing: roughly 10 US cents per 30 seconds of streaming (Full mode), or per minute (Lite mode).

B.5.4 Hospital Backend Connection

- Stand up the mock Doctor / Directions / Service-Price / Appointment APIs as scoped for Sprint 0 (Section 15.1), so Alexa has something real to call from day one.
- Put an API gateway in front of these mocks now — the same gateway real HIS/scheduling integrations will sit behind later.
- Wire appointment requests through as an MCP tool call rather than a plain REST call (Section A.4), so the appointment sub-task is already shaped the way a future multi-agent layer would expect it.

B.6 Small Demo (Proof-of-Concept) Scope

A focused, two-to-three-journey demo is enough to validate the stack and surface the orchestration gap:

Journey	What it proves	Data source
Welcome + language selection	Kiosk/web shell, avatar idle/greeting state, Arabic/English switch	Static config
Doctor search → directions ("Where is Dermatology?")	Full turn shown in Figure B.2: STT → intent → mock API → TTS → avatar answer	Mock /departments API

Journey	What it proves	Data source
Appointment request (single specialty, no real booking)	MCP tool call pattern end-to-end; where the gap is most likely to show	Mock /appointments API

Deliberately out of the demo: WhatsApp handoff, real patient verification, and any live HIS/scheduling data — matching the MVP Pack's own Day 16–30 “build foundation” scope.

B.7 Cost and Latency Notes

Component	Pricing model	Latency profile	Notes
Alexa	Platform subscription (free tier available; paid tiers for volume/channels)	Orchestration overhead only — not the bottleneck	No-code flow builder; JS/webhook nodes for custom logic
ElevenLabs	Billed per character (TTS) / per audio minute (STT); free tier ≈ 10k characters/month	Flash v2.5: well under 500ms end-to-end	Reserve higher-quality models for offline content
HeyGen LiveAvatar	Credit-based: ~10 US cents / 30 sec (Full) or per minute (Lite)	Real-time WebRTC streaming	Separate credit pool from HeyGen's standard video plans

These figures are enough to size a pilot-month estimate once Operations confirms expected daily conversation volume.

B.8 Known Gap — Reconfirmed, Not Solved

Reminder: This plan operationalizes Option 1 as-is; it does not add multi-agent coordination. The appointment-request demo journey (Section B.6) is specifically chosen because it is the most likely place the gap shows up. Watching where the flow-builder logic gets awkward is the actual output of this demo, feeding into the Option 1 vs. Option 2 decision in Section A.10.

PART C

Option 3 — Open-Source Graph-Memory Real-Time Avatar

Built on Graphiti — a self-hosted temporal knowledge graph engine

C.1 What This Option Is

This option was surfaced from a public engineering tutorial (Akshay Pachaar / "Daily Dose of Data Science", ai-engineering-hub repository) demonstrating a real-time AI avatar with "human-like memory." The distinguishing idea is not the avatar rendering itself, but the memory layer underneath it: instead of standard vector-similarity RAG, it uses Graphiti — an open-source (Apache 2.0), self-hostable temporal knowledge graph engine built by the Zep team.

Unlike Options 1 and 2, which are about how conversations are orchestrated, this option is about what the assistant remembers and how current that memory stays. It is best understood as a candidate upgrade to Section 9 (RAG) of the MVP Pack, not a replacement for Alexa or the avatar/speech layer — it can sit underneath either Option 1 or Option 2.

C.2 Why a Temporal Knowledge Graph Instead of Vector RAG

Standard vector RAG retrieves the most similar chunks of approved text to a patient's question. It works well for static content (FAQs, policies) but has a known weakness: when a fact changes — a doctor moves clinics, a price is updated — the old chunk and the new chunk can both look similar enough to the query to be retrieved, with no built-in concept of which one is currently true.

A temporal knowledge graph stores facts as relationships between entities, each with a validity window. Updating a fact adds a new, currently-valid relationship and marks the old one as superseded, rather than requiring deletion. This matches doctor-to-clinic assignments, price/package changes, and department relocations directly — exactly the facts that must never be answered from stale data (Section 8.1).

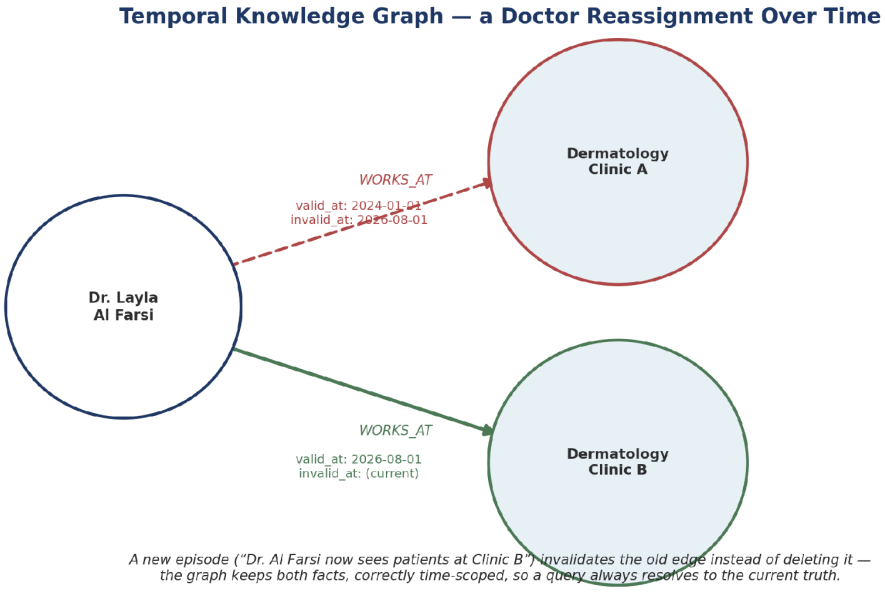


Figure C.1 — A doctor's clinic reassignment as a temporal graph: the old assignment is kept but marked superseded, not deleted.

C.3 Core Components

Layer	Role	This option
Orchestration	Channel handling, flow logic, calling out to memory and hospital APIs	Alexa (unchanged from Option 1/2)
Memory / knowledge layer	Stores and retrieves approved hospital facts	Graphiti — temporal knowledge graph, self-hosted
Graph database	Stores the entities/relationships Graphiti manages	Neo4j, FalkorDB, or Amazon Neptune
Extraction LLM	Reads new content and proposes entities/relationships into the graph	Any OpenAI-, Anthropic-, Gemini-, or Groq-compatible model
Speech + avatar	Voice and rendered video, as in Option 1	Reusable as-is — ElevenLabs / HeyGen, or open alternatives

C.4 Architecture

The overall shape stays the same as Option 1 — Alexa remains the single orchestration hub talking to speech, avatar, and hospital APIs. The only structural change is what sits behind the knowledge-lookup call: a self-hosted Graphiti/graph-database layer instead of (or in addition to) the vector RAG store.

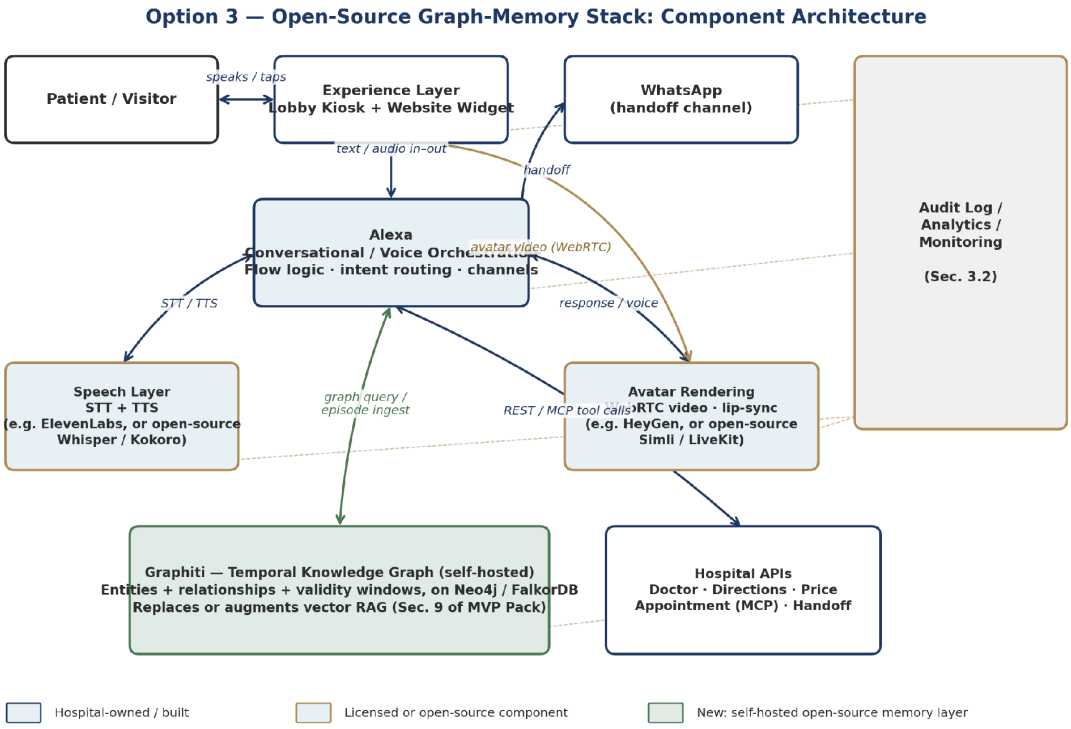


Figure C.2 — Option 3 architecture: the green box is the new self-hosted memory layer; everything else can be reused from Option 1.

Because Graphiti is self-hosted, this is the option that sits most comfortably inside the hard requirement in Section A.6 — patient-relevant facts never have to leave hospital-controlled infrastructure.

C.5 Code Walkthrough — Ingesting a Hospital Fact

Graphiti's core unit of information is an "episode." Below is an illustrative example of how a doctor-directory change would be ingested, following Graphiti's published quickstart pattern:

```
from datetime import datetime, timezone
from graphiti_core import Graphiti
from graphiti_core.nodes import EpisodeType

graphiti = Graphiti(
    uri="bolt://localhost:7687",      # hospital-hosted Neo4j / FalkorDB
    user="neo4j",
    password=os.environ["GRAPH_DB_PASSWORD"],  # from hospital secrets vault
)
await graphiti.build_indices_and_constraints()   # one-time setup

await graphiti.add_episode(
    name="doctor_directory_update_2026_08_01",
    episode_body=(
        "Dr. Layla Al Farsi now sees patients at Dermatology "
        "Clinic B, Level 3, Block C. She no longer holds "
        "clinic hours at Clinic A."
    ),
    source=EpisodeType.text,
    source_description="Approved doctor-directory update, Medical Affairs",
    reference_time=datetime(2026, 8, 1, tzinfo=timezone.utc),
)
```

Graphiti's extraction step reads that sentence, recognizes the doctor and clinic entities already in the graph, adds a new "works at Clinic B" relationship valid from 2026-08-01, and automatically marks the older relationship as no longer current — without anyone writing that logic by hand.

C.6 Code Walkthrough — Answering a Patient Question

On the query side, Alexa would call a single search against the graph rather than a vector similarity lookup:

```
results = await graphiti.search(
    "Where does Dr. Layla Al Farsi see patients?"
)

for r in results:
    print(r.fact, r.valid_at, r.invalid_at)

# Example output:
# "Dr. Layla Al Farsi works at Dermatology Clinic B"
#    valid_at=2026-08-01   invalid_at=None   (current)
```

Graphiti's search combines semantic similarity with keyword matching and graph-distance reranking, and returns facts tagged with their validity window — so the response passed back through Alexa to ElevenLabs/HeyGen only ever contains the currently valid fact, satisfying the same guardrail as the existing vector-RAG design (Section 8.1).

C.7 Guardrail Fit

Reminder: Graphiti changes how facts are stored and retrieved — it does not relax any guardrail. The response text returned to Alexa must still come only from a graph query result, never from the extraction LLM's general knowledge.

C.8 Vector RAG vs. Graph RAG — Comparison

Dimension	Vector RAG (current MVP Pack, Sec. 9)	Graph RAG — Graphiti (Option 3)
How facts are found	Similarity between the question and text chunks	Relationships between named entities
Handling changed facts	Old and new chunks can both look relevant; needs manual re-indexing	New episode automatically supersedes the old relationship; both kept, time-scoped
Best suited to	Static content: FAQs, policies, service descriptions	Facts that change on a schedule: doctor assignments, prices, department moves
Hosting	Any vector DB, can be managed/external	Self-hosted graph DB — stays inside hospital infrastructure
Maturity for this project	Already scoped and understood (Section 9)	New — needs a proof-of-concept first

These are not mutually exclusive: a pragmatic path is to keep vector RAG for static content and add Graphiti specifically for the entities that change — doctor directory, price catalogue, department assignments.

C.9 Where This Sits Relative to Options 1 and 2

This is not a competing orchestration path — it is a memory-layer upgrade that can sit underneath either Option 1 or Option 2. It does not solve the multi-agent coordination gap identified in Section A.10; a graph memory still needs to be called by whatever orchestration layer wins that decision. Its main contribution is to the RAG requirement (Section A.5) and the data-control requirement (Section A.6).

C.10 Maturity and Risk

- Origin is a public tutorial/reference implementation, not a supported product — expect to invest real engineering time to harden it (error handling, access control, backup/restore of the graph database).
- Requires running and maintaining a graph database (Neo4j/FalkorDB) in the hospital's private/hybrid environment — an additional operational component versus a purely licensed vector-RAG service.
- Ingestion makes an LLM call per fact update — worth costing out for the expected volume of doctor/price/department changes per month.
- Extraction quality (recognizing the same entity every time, in Arabic and English) needs its own test set, similar in spirit to the AI Quality Evaluation Set (Section 8.4).

PART D

Option 4 — Avatar Rendering Alternatives

A swappable avatar-rendering layer evaluated against HeyGen LiveAvatar

D.1 Why Re-Open the Avatar Choice

Section A.10 named HeyGen LiveAvatar as the avatar-rendering component of Option 1, mainly because it was ready out of the box and integrates natively with ElevenLabs voices. Since then, the avatar-rendering market has broadened noticeably — LiveKit's plugin ecosystem alone now lists more than a dozen real-time avatar providers. This section treats avatar rendering as its own, swappable decision, separate from the orchestration (Alexa) and speech (ElevenLabs) choices, which do not need to change.

D.2 Three Categories of Avatar-Rendering Approach

Category A — Modular "face-only" APIs

You keep your own voice/orchestration stack (Alexa + ElevenLabs) and send audio or text to the provider, which returns a real-time, lip-synced video stream. Most natural fit since orchestration is already decided. Candidates: Simli, D-ID (Agent API), Anam, Tavus (Conversational Video Interface), and HeyGen LiveAvatar itself.

Category B — Fully-rendered 3D digital humans

Providers build a complete, often photoreal 3D character with full creative control over appearance — closer to the culturally specific Emirati persona in the original concept art. Candidates: Soul Machines, RAVATAR. Typically more production investment and can carry higher latency than Category A.

Category C — Fully custom / self-hosted build

Build the avatar in-house using a 3D engine (Unreal Engine MetaHuman) and a real-time rendering/streaming pipeline (e.g. NVIDIA ACE / Omniverse Avatar Cloud Engine). No per-minute vendor billing once running, and the persona and all rendering stay on hospital-controlled infrastructure — consistent with Option 3's self-hosted philosophy — but with real engineering and GPU-hosting investment.

D.3 Where It Plugs Into the Architecture

Whichever provider is chosen, it sits in exactly the same place Figure B.1 already reserved for HeyGen: Alexa sends response audio/text to the avatar layer and receives a WebRTC video stream back for the kiosk/web display. Nothing else in the architecture changes.

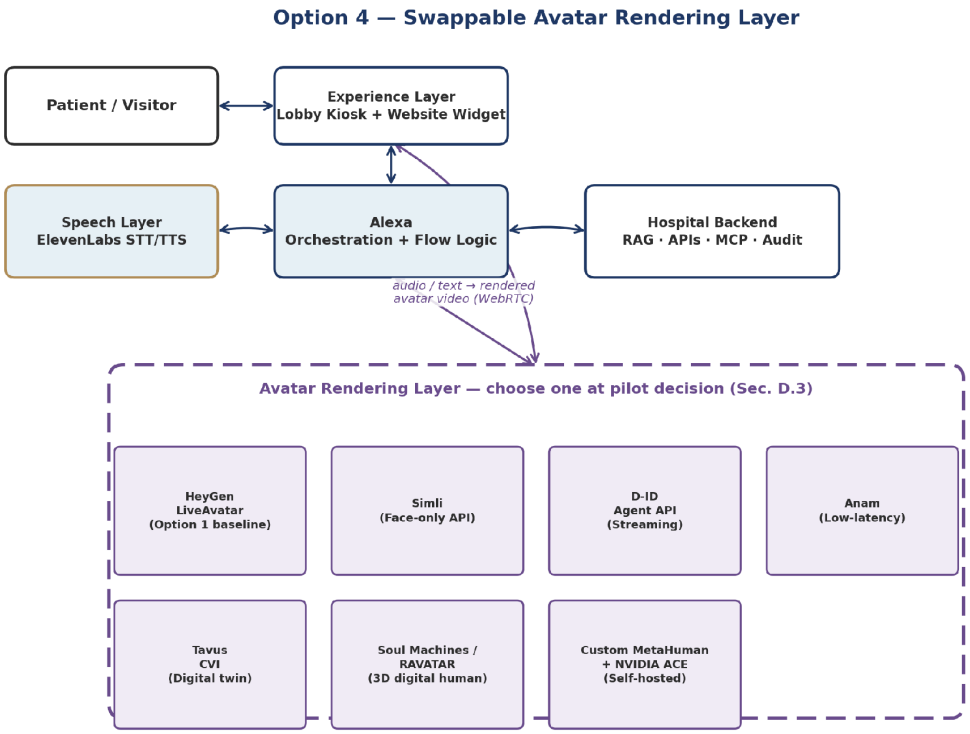


Figure D.1 — The avatar-rendering layer as a swappable component behind Alexa; every candidate plugs into the same interface.

D.4 Candidate Comparison

Provider	Category	Latency / streaming	Indicative cost	Custom persona support	Notes
HeyGen LiveAvatar	A	Real-time WebRTC	~$0.10 / 30 sec (Full)	Yes — from photo/video	Option 1 baseline; native ElevenLabs voice support
Simli	A	Real-time, speech-to-video	Usage-based	Yes	Most "unbundled" — pure face renderer, no bundled voice/LLM
D-ID (Agent API)	A	WebRTC, sub-2s first frame	Usage-based	Yes — animates a photo	One of the more mature developer ecosystems
Anam	A	~180ms benchmark (CARA-3)	Usage-based	Yes	Newer entrant; worth testing for responsiveness
Tavus (CVI)	A	Real-time conversational video	Usage-based	Yes — "digital twin"	Strong fit if a highly personalized persona matters
Soul Machines / RAVATAR	B	Higher than Cat. A, real-time capable	Higher — production pricing	Full creative control	Closest to the concept art's photoreal look
Custom: MetaHuman + NVIDIA ACE	C	Depends on hospital GPU hosting	~$0.06–$0.12/min raw compute	Full control	Largest upfront effort; best long-run data control

As general market context: managed real-time avatar APIs (Category A) typically run $0.10–$0.37 per minute depending on resolution and concurrency, while a self-hosted custom build (Category C) can bring raw compute cost down to roughly $0.06–$0.12 per minute — the gap being convenience and operations, not underlying rendering quality.

D.5 Evaluation Criteria Specific to Al Salam

Criterion	Why it matters here
Custom persona fidelity	The concept art specifies a culturally appropriate Emirati-male persona in hospital-branded dress — the provider needs to reproduce that, not offer a generic stock character.
Arabic lip-sync quality	Same bar as the speech layer (Section B.5.2) — needs testing with real Arabic hospital terminology, not just English demo scripts.
Latency in a lobby setting	A kiosk conversation feels broken quickly if there's a visible delay between voice and mouth movement; the main practical differentiator between Category A providers.
Data control	Categories A and B send response content to an external service every turn; Category C keeps it entirely on hospital infrastructure (Section A.6).
Integration effort vs. Sprint 0 timeline	Category A providers plug into the existing pipeline within the 90-day window; B and C carry meaningfully more production/engineering time.

D.6 Suggested Approach

Rather than a single up-front choice, this fits the report's existing pattern of validating options against real journeys before committing (Section A.10): trial two or three Category A providers (HeyGen LiveAvatar plus one or two of Simli / D-ID / Anam) against the same doctor-search and directions journey used in the Option 1 demo (Section B.6), scoring them on the criteria in Section D.5. Categories B and C are worth keeping on the roadmap as a post-pilot upgrade path rather than as MVP candidates.

PART E

Master Action Plan & Decision Owners

Every open decision from Parts A–D, deduplicated into one timeline

Action	Owner	Needed by	From
Approve MVP scope, pilot location, and this report's four-option evaluation approach	Executive Sponsor / Product Owner	Day 1	A
Provision Alexa workspace, ElevenLabs API key, HeyGen LiveAvatar credits	Backend/Integration + DevOps	Day 3–5	B
Score Humanizing and shortlisted avatar/speech vendors in the Section 14 evaluation matrix	IT / Procurement	Day 5–10	A
Score HeyGen LiveAvatar, Simli, and D-ID against Section D.5 criteria using the Option 1 demo journey	AI/RAG Engineer + Frontend/Kiosk Dev	Day 5–10	D
Request a custom-persona sample (matching the concept art) from each shortlisted avatar provider	IT / Procurement	Day 5–10	D
Decide whether lightweight (non-AR) wayfinding is added to MVP backlog or deferred to Epic E8	Product Owner / Operations	Day 10	A
Build the 3-journey demo flow against mock APIs (Section B.6) and validate Option 1 vs. Option 2	AI/RAG Engineer + Dev Lead	Day 10 (M1)	B
Record where the appointment-request journey strains the flow builder, feeding the Option 1 vs. 2 decision	AI/RAG Engineer	Day 10 (M1)	B
Draft pilot-month cost estimate using Section B.7 unit prices + Operations' expected volume	PM/Business Analyst + Operations	Day 10	B
Get a rough production quote from one Category B avatar provider (Soul Machines or RAVATAR) for comparison	IT / Procurement	Day 10	D
Stand up Graphiti + Neo4j (or FalkorDB) in DEV; ingest the current doctor directory as a test set	AI/RAG Engineer	Day 10–15	C
Confirm hosting/licensing plan for the chosen orchestration framework within the private/hybrid architecture	IT Architecture / Security	Before M2 (Day 30)	A
Run the doctor-reassignment scenario (Sections C.5–C.6) end-to-end; confirm old facts are correctly superseded	AI/RAG Engineer	Before M2 (Day 30)	C
Compare Graphiti answer quality/latency against the existing vector-RAG plan for the same 20 test questions	AI/RAG Engineer + Quality	Before M2 (Day 30)	C
Decide RAG scope: full replacement of vector RAG, or hybrid (graph for changing facts, vector for static content)	Product Owner + AI/RAG Engineer	Before M2 (Day 30)	C
Flag Category C avatar build (custom MetaHuman) as a post-pilot roadmap item, not an MVP candidate	Product Owner	M1 decision record	D

"From" refers to the report part each action originated in (A = Concept & Approach, B = Option 1, C = Option 3, D = Option 4), for traceability back to the fuller discussion.